School of Computer Science and Engineering, UNSW

Never Stand Still

WK08-02: Characteristics & Limitations: Functionality, Performance, and Reliability

Software Architecture for Blockchain Applications, COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

Functional suitability

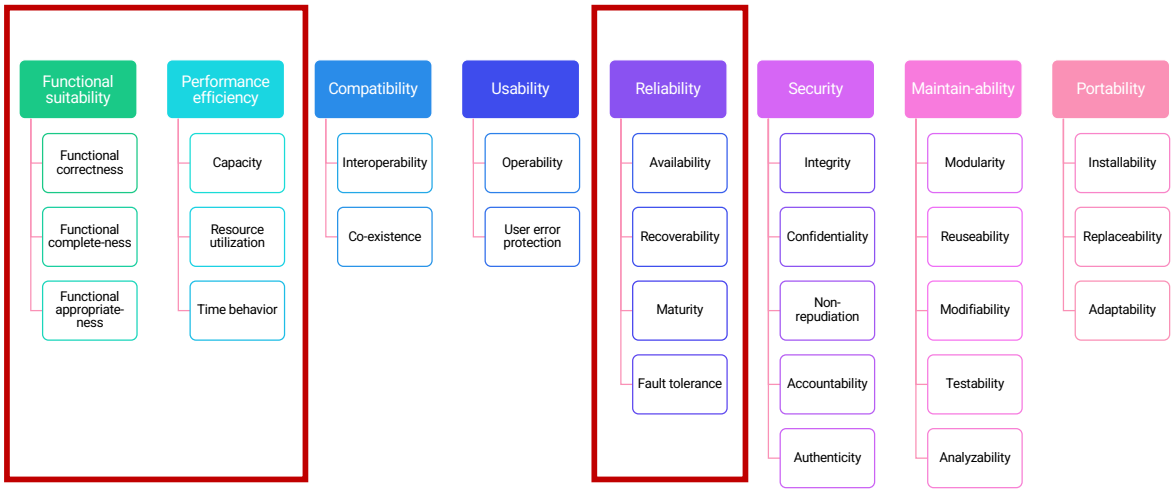
Performance efficiency

- Time behaviour; latency and throughput

Reliability; availability and recoverability

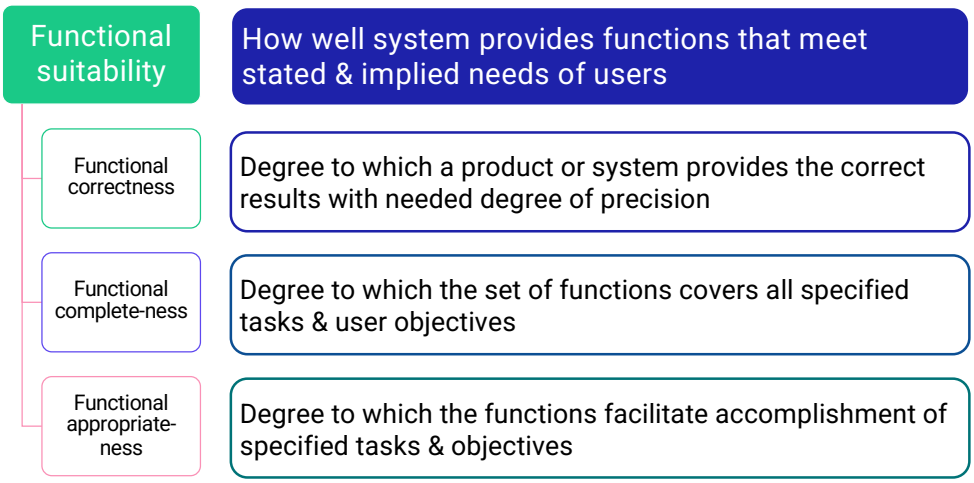
- We'll start with an introduction to functional suitability, though it is more about functional requirements.
- Then, we discuss functional performance efficiency. However, we focus only on latency and throughput which characterises the time behaviour of a system.
- Time behavior is the measurement of an application's ability to react to different events over time, encompassing the duration required for the application to perform particular actions such as loading a page, running a query, or processing a transaction. Time behavior also includes evaluating how the application performs under various loads, including high traffic or heavy user activity.
- Finally, we discuss availability and recoverability under reliability

ISO/IEC 25010:2011 Quality Model

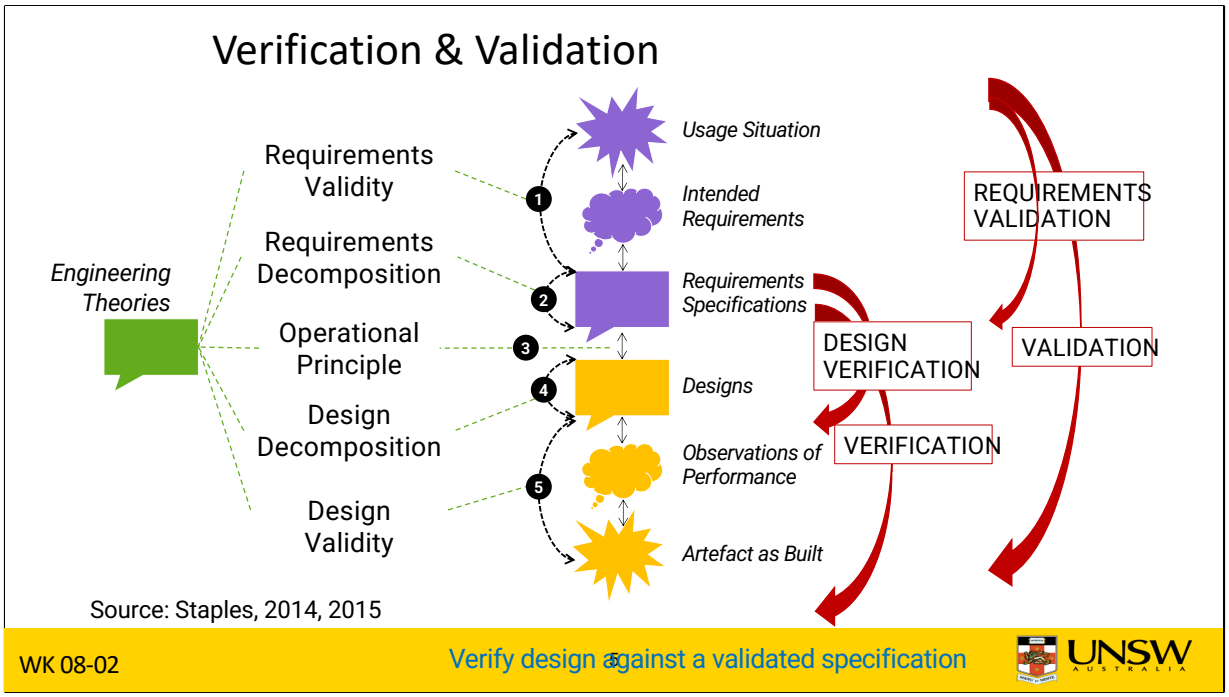


- International standards for evaluation of software quality, ISO/IEC 25010:2011 Quality Model, define many software qualities.
- Other than functional suitability all other qualities are non-functional. Under each group, there are multiple quality metrics.
- In this topic, we'll focus on a subset of properties under the 3 highlighted ones.

Functional Suitability



- Functional suitability represents how well the product or system provides functions that meet the stated and implied needs of the users.
  - These needs are typically stated in the requirements specification.
  - Functional suitability focuses on functional requirements while all other qualities in the ISO quality model are related to non-functional requirements.
- ISO quality model focuses on functional aspects like correctness, completeness, and appropriateness.
- Functional correctness is about to what extent a product or system provides the correct results with the needed degree/level of precision.
  - A fingerprint-matching algorithm may find a match with a particular confidence score. Also, it may have false positives and false negatives. The question is whether the numbers are appropriate for use.
- Whereas functional completeness is about the degree to which the set of functions covers all specified tasks we want to achieve as per our user objective. This is about the ability to meet all requirements.
  - E.g., covering 95% of the requirements.
- Functional appropriateness is the degree to which the functions facilitate the accomplishment of specified tasks and objectives. It's about the ability to achieve what you want.
  - A fingerprint algorithm used for access control may not be suitable for providing evidence for a court case to prove a match given a large database of fingerprints.



- To determine the functional suitability of a system, we need to verify it against the specification.
- While we design a system based on requirements, requirements can be underspecified and changed. Thus, we need to validate both the requirements and design throughout.
    - See the 2 sets of arrows on the right. One indicates the idea what is built is verified against the specification. The other one validates the requirements throughout.
- This is how we go about doing it:
    - Requirements Validity: What are the needs for usage situations, and how are they specified?
    - Requirements Decomposition: How are requirement specifications decomposed?
    - Operational Principles: Does a design satisfy a requirements specification?
    - Design Decomposition: How are designs decomposed?
    - Design Validity: What are artefacts & behaviours, and how are they represented in designs?

What Should Your Specifications Be?

"The DAO" failure on Ethereum in 2016 shows that

- Code was stated as the specification
- There was no problem with EVM
- Code functioned exactly as it was written
- Was reentrancy a "bug" in the DAO code?

Lessons from the DAO failure

- Code is not the law
- Code is not a good way of specifying smart contracts

Open question: How should we specify smart contracts?

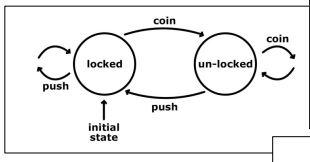
- The previous slide says, while the design is from the speciation, we also need to question the validity of the specification. So, the question is what should be the specification?
- The DAO (Decentralised Autonomous Organisation) attack is a classic example of reliance only on code and governance issues around it. The DAO was launched in April 2016 as a decentralised venture capital fund.
    - DAO code was stated as the specification.
    - In the DAO case, EVM was not the problem, it worked as expected, where a call can come back to the caller function or smart contract. This is called reentrancy.
    - Given the code is the specification, it functioned as exactly it was written.
    - So, the question is reentrancy a "bug" in the DAO code? It's a feature of EVM used in the wrong way. Anyway, it led to a slow leak of > $50M.
- We can learn a few things from DAO failure:
    - While Ethereum classic considered "Code is law" Ethereum bend the law by performing a hard fork. Of course, one can argue that the law needs to evolve.
    - It also tells us that code isn't a good way of specifying smart contracts.
- Open research question: How should we specify smart contracts?

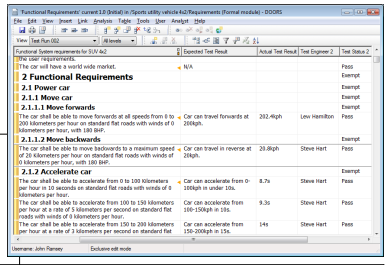
Specifications are Models of Requirements

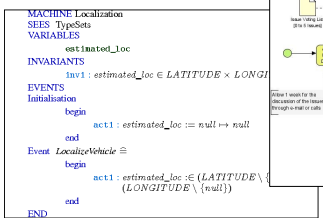
Natural language specifications
- Lists of requirements
- Use cases & Scenarios

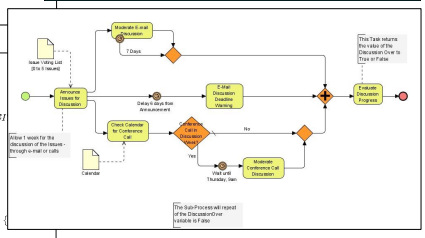
Models
- State machines
- Business process models

Formal specifications
- Z, Object-Z, B, Event-B, HOL









- A specification is some form of a model(s) of requirements. Here are a few ways to specify smart contracts.
- You are already familiar with natural language specifications like a list of requirements, use cases, and scenarios. Think about the requirements you stated in your course project.
- Modelling is another way. We can include a state machine or a flow diagram as a model of how the system should exhibit its dynamic behaviour. There are other ways like business process models illustrated using BPMN (Business Process Model and Notation).
- An even more precise approach is formal specifications.
    - Z notation is a formal specification language used for describing and modelling computing systems.
    - Object-Z is an object-oriented extension of Z.
- Natural language statements are the easiest forms of specification and hence are the most common. They are usually supplemented with models like state machines or BPMN diagrams.
- While it's difficult to specify a large program in formal specifications, these are the most precise.

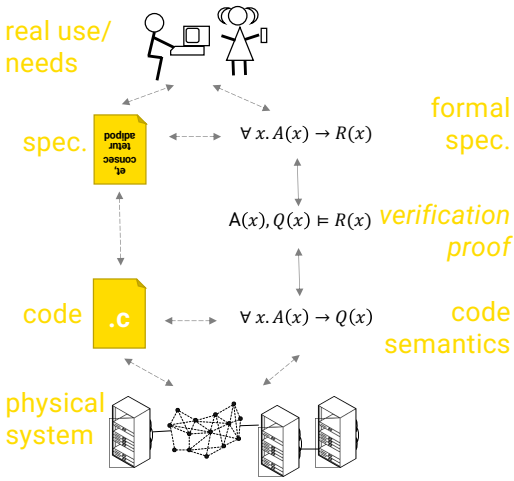
Specifications are Models of Requirements

Program testing can be used to show the presence of bugs, but never to show their absence! –Dijkstra, '60s/'70s

Formal methods are logical reasoning about all possible behaviours of a software

Is the strongest kind of evidence that the software is correct

- Precise, but hard
- Smart contracts aren't that big?
  - Cardano formally verified
  - Ongoing efforts to formally verify certain aspects of Ethereum



- These slides give an overview about how we can formally specify and verify a system or a smart contract.
- As early as the 60s and 70s Dijkstra said "*Program testing can be used to show the presence of bugs, but never to show their absence!*"
- So, we need a different approach to ensure a program is free of bugs or at least minimum of bugs, and formal methods are the best around.
- Formal methods are logical reasoning about all possible behaviours of software.
- This is the strongest kind of evidence that the software is correct.
  - While it's difficult to specify a large program in formal specifications, smart contracts are small enough for such modelling.
  - Even the Linux kernel was formally verified by a joint effort between Data61 and UNSW.
  - Cardano is formally verified, see https://docs.cardano.org/
  - There are some efforts to formally certain aspects of Ethereum. See https://github.com/leonardoalt/ethereum_formal_verification_overview
- The figure illustrates the formal verification process.
  - First, you come up with a formal specification which is more like a set of mathematical statements. You need to make sure this matches with the natural language or other forms of the specification.
  - Then you come up with a set of proof to prove the properties you want.
  - Then you talk about a formal representation of the implemented source code.
  - Then you use the verification proof to prove whether the code semantics satisfy or do not satisfy the formal specification.
  - In this process, you should also need to consider formal specifications of the underlying computing system like the CPU or the EVM.

Agenda

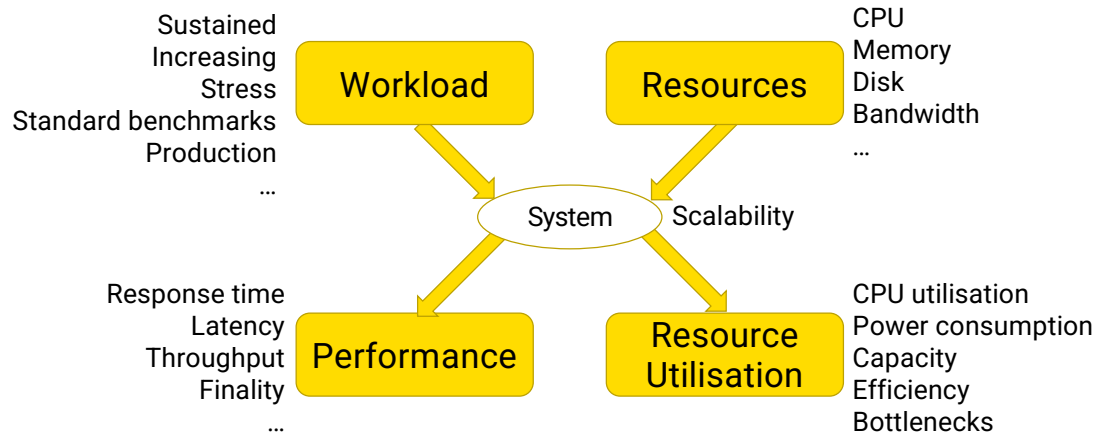
Functional suitability

Performance efficiency

- Time behaviour; latency and throughput

Reliability, availability and recoverability

Performance Related Considerations



- This figure captures performance-related considerations of a software system.
- We provide a system while providing resources like CPU (speed, number of cores), memory, storage, and bandwidth.
- The system is expected to handle a certain load (aka workload). Workload characteristics include user requests, how frequently they arrive, the size of requests, or the complexity of those requests.
    - E.g., if we consider workload on a web server, this includes HTTP requests, how frequently they arrive, the size of an HTTP request in bytes, and whether these requests are to read or write data.
    - With time workload is likely to increase.
    - Sometimes, the system may receive a very high load (aka flash crowd), e.g., a website selling event tickets may attract a very high number of HTTP requests as soon as they open sales of a popular sports event or concert.
    - There are standard benchmarks to test a system's performance, e.g., TPC-C (Transaction Processing Performance Council Benchmark C) benchmark is used to compare the performance of online transaction processing (OLTP) systems. While benchmarks are good to compare 2 systems, they don't necessarily represent the workload mix of a real system.
- Given the resources and workload, we can measure different aspects of the performance of a system.
    - For web-based applications, we are typically interested in response time, latency, and throughput. For blockchain-based applications, we are also interested in time to finality.
- Another measure is how much of the assigned resources are used by the system, which is called resource utilisation.
    - E.g., these include CPU, memory, disk, and bandwidth utilisation. While low utilisation means over-provisioning of resources high utilisation is also not desirable as the system may not have additional capacity to handle increasing workloads on short-term stress on the system
    - Power consumption is another measure.
    - Resource utilisation can also be measured as efficiency, i.e., how efficient the system is using allocated resources.
    - While low resource utilisation is usually due to a low workload, it may also indicate bottlenecks in the system. E.g., when one component of the system is highly utilised and cannot cope with the workload, other components may remain idle as the bottleneck component cannot finish its task and pass the data/request to other components.
        - E.g., When disk access is slow system may spend more time on IO reducing CPU utilisation.
- Scalability is an orthogonal aspect that concerns on the ability of a system to sustain performance with an increasing workload. Usually, to retain performance, you need to increase resources. However, the workload to resource need relationship may not be linear in many cases, e.g., x amount of increase in workload may need x^2 resources to maintain the same performance.

Blockchain Workloads & Resources

Workloads

- Cryptocurrency TXs
  - UTXOs in a TX, TX inter-arrival time
- Smart contract deployment & execution
  - Size of contract, TX data size, & complexity of SC functions

Resources

- Public blockchains
  - Node & mining hardware & bandwidth
  - Mining difficulty in PoW, Staked cryptocurrency in PoS
  - Block & TX gas limit
  - TX fees
- Private blockchains
  - Typical cloud/enterprise server & network resource considerations

- Blockchain workloads and resources are a bit different from typical software systems. This slides capture some of those differences.
- The workload on a blockchain depends on cryptocurrency transactions and transactions invoking smart contracts.
  - A transaction with multiple UTXOs will involve a bit more work than one with a single UTXO. Whereas in the account-balance model, a transaction transferring 1000 ETH has the same complexity as one transferring 0.0001 ETH.
  - However, workload increases when transactions arrive together where many transactions may accumulate in the transaction pool. This behaviour can be measured using the inter-arrival time of transactions.
  - Transactions deploying or invoking smart contracts can have a complex mix of workloads. E.g., a larger smart contract consume more resources to transfer and deploy it. Similarly, a transaction that includes more data would require more bandwidth and storage. However, the biggest variability comes from the complexity of smart contact functions where the resources to execute them depend on both input data and business logic.
- As with any software system blockchains also include computational and storage resources.
  - However, when it comes to a public blockchain we do not typically worry about node hardware including specialised mining hardware. As these affect the miners/validators, not the users.
  - Other user-configurable forms of resources on public blockchains include mining difficulty in PoW blockchains and staked cryptocurrency in PoS.
  - Other configurable resources are block and transaction gas limits in Ethereum.
  - Transaction fees are the most visible resource to the users which captures all miner's efforts including hardware costs.
  - Private blockchain resources are similar to typical cloud or enterprise server and network resource considerations.

Important Varieties of Performance

Latency	• How quickly does the system respond to a request?
Throughput	• How many requests can the system process in a unit of time?
Timeliness	• Will the system always process a request in a specified time window?
Scalability	• An orthogonal issue • How does performance change under increasing workload & system resources?

- Here are some of the important varieties of performance, we have already discussed some of them like latency and throughput.
- Latency is how quickly does the system respond to a request? For e.g., the time taken to include a TX in a block
- Throughput is how many requests can the system process in a unit of time? For e.g., we say Bitcoin can include 7 TXs per second
- Timeliness is about a system's ability to always process a request in a specified time window? For e.g., blockchains usually don't provide any guarantees on how soon a TX a will be included in a block
- Scalability is an orthogonal issue. It can be considered as how does performance change under increasing workload & system resources?
    - It can also be considered as a system's ability to retain performance with the increasing workload

Factors Affecting Performance of Blockchains

Block size in bytes or block gas limit

Inter-block time
- Block difficulty in PoW blockchains
- Configurable in others

TX finality
- Nakamoto consensus – Wait for multiple confirmations due to probabilistic finality
- Super majority – Immediate finality

No of pending TXs

TX fees in public blockchains

Other factors
Complexity of smart contracts & ledger state accessed, e.g., Hyperledger Fabric

TX & ledger privacy settings, e.g., Hyperledger Fabric

Hardware capacity – For enterprise blockchains

Network capacity, including speed of light for global TXs & block propagation

Maximum waiting time in TX pool, minimum gas price, altruistic miners, …

- Many factors affect the performance of blockchains. In the next set of slides, we discuss some of them.
- We already discussed the impact of the block size (either in bytes or block gas limit) and inter-block time on TX throughput
  - The larger the block size, more TXs it could include
  - The smaller the inter-block time, it can build more blocks within a unit of time
  - Inter-block time in PoW blockchains is determined by the block difficulty
  - Whereas in other inter-block time is configurable, e.g., Ethereum 2 plans to emit a block every 12 seconds. Inter-block time can be configured in Hyperledger Fabric
- TX finality is another factor that affects latency and timeliness
  - If the blockchain uses Nakamoto consensus, we must wait for multiple confirmations due to probabilistic finality. Once the TX is included in a block, more subsequently added after that ensure whether the TXs is in a block that is in the longest/heaviest chain\
  - When the consensus is based on the super majority, TX is typically considered to be final as soon as the majority accept the block containing it as valid. In such cases, we refer to as “immediate finality”
- Another factor is the no of pending TXs. Blockchain is essentially a demand-supply system with limited supply. When more TXs accumulate in the TX pool (i.e., high demand), it will increase the waiting time for a TX as supply is limited by block size and inter-block time
- TX fees in public blockchains is a key factor that determines the latency and timeliness experienced by a single TX
- There are several other factors
  - It has been demonstrated that in enterprise blockchains like Hyperledger Fabric (HLF), complexity of smart contract functions and ledger state accessed tends to determine the throughput and latency. For e.g., TXs in HLF need to be first endorsed. So if the TX is more complex to execute, it will slow down endorsement process. Also, as HLF uses a read-write set to validate a TX between execute, order, and validate steps, if multiple TXs depends on the same state, read-write set for a TX may be inconsistent between execute and validate steps. Consequently, TX will fail and need to retry
  - It has also been demonstrated that TX and ledger privacy settings in Hyperledger Fabric show down TXs as further validations need to be performed and ledger is accessed/updated from/at a limited set of nodes
- Hardware capacity like CPU speed, number of CPUs, and memory are also key factors that determine the performance of enterprise blockchains. They do have an impact on public blockchains, but difficult get adjusted in proportion to global computing power
- Network capacity (both bandwidth and speed of light) is another factor that determines how fast a TX/block can propagate in a global network
- There are many other configuration parameters that may affect the performance experienced by a single TX. For e.g., in Ethereum maximum waiting time in TX pool, minimum gas price, and altruistic miners have an impact on (low fee paying) TXs

To Understand Performance, You Need Measurements

Application-level measurements
- Overall performance; check requirements met

Component-level measurements/ Micro-benchmarking
- Piece-wise drivers of performance; observe interactions; identify bottlenecks

Types of performance testing
- Load testing – Check system response & utilisation under increasing load
- Stress Testing – Check maximum capacity; find resource bottlenecks
- Soak Testing – Check system response under long-duration load

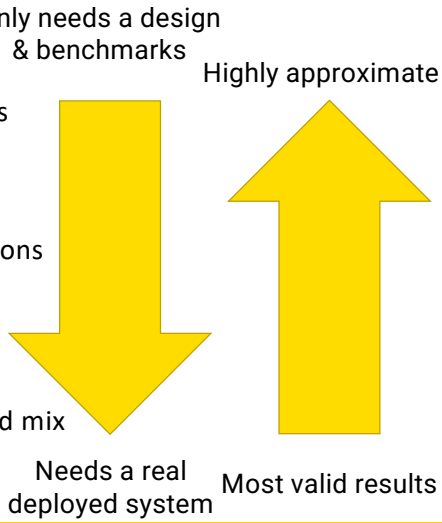
- To understand the performance, you need measurements. Either you measure the real system or use some learnings from related systems or simulations.
- We can measure performance at different levels, eg.,:
  - Application-level measurements focus on overall performance. It can help check requirements are met.
  - Component-level measurements (aka Micro-benchmarking) allow piece-wise drivers of performance. This allows us to observe interactions among components and identify bottlenecks more easily than in testing the entire system. However, we can't derive overall system performance just by somehow aggregating the performance of components.
- Different types of performance testing can be used to estimate performance and demonstrate different targets, e.g.,
  - Load testing checks system response and resource utilisation under increasing load.
  - Stress Testing checks the maximum capacity of a system to find resource bottlenecks.
  - Soak Testing checks system response under a long-duration of load to see whether the system can sustain the load for a long time.

Predicting Performance

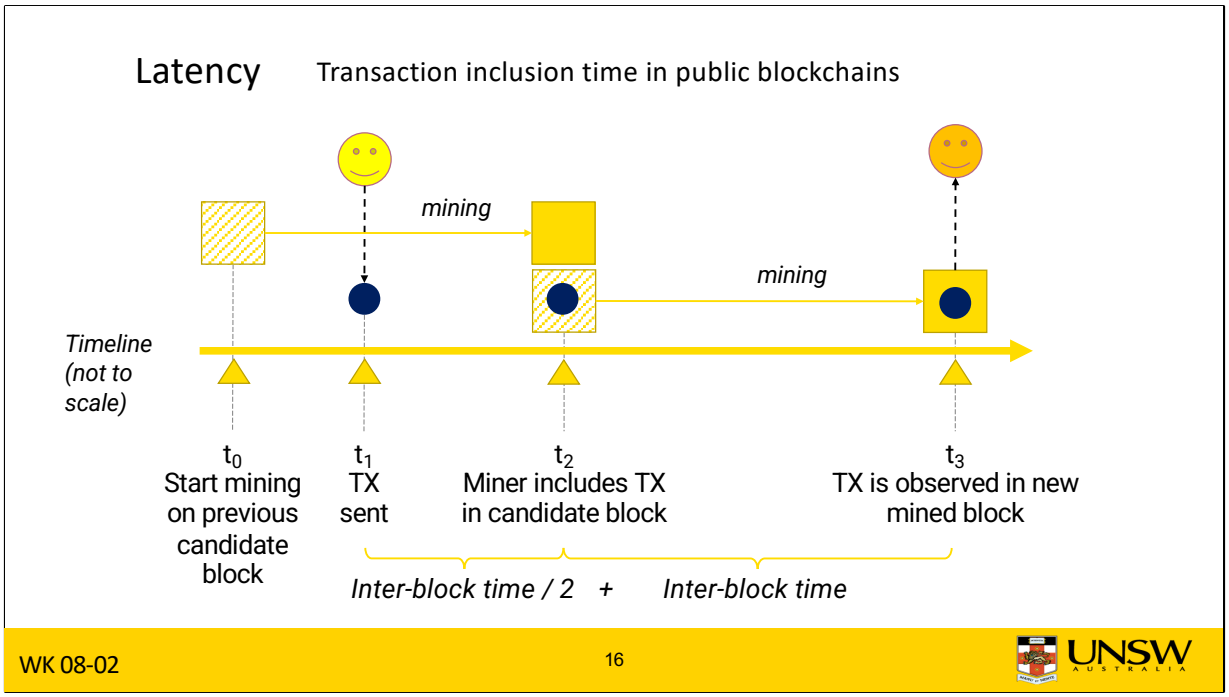
Analytic formulae
- Highly abstract/simplified models
- Only needs design & component benchmarks
- Quick & dirty

Simulation
- Uses a model; many abstractions/simplifications
- Only needs a design & benchmarks
- Can explore variation & interactions

Monitoring
- Needs real deployed system & good workload mix
- Needs a monitoring framework
- Most valid results

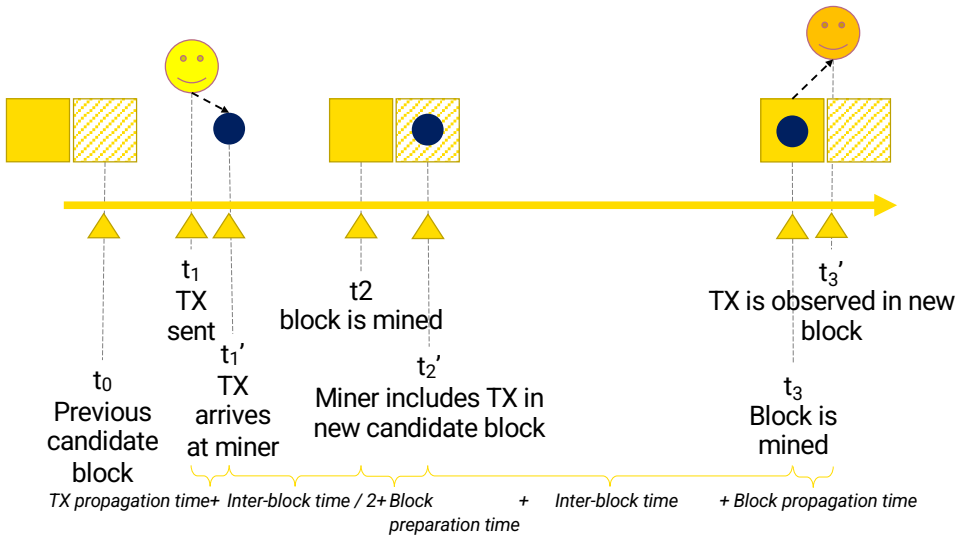


- While we need the implemented system to test its full performance and fine-tune it, we need to be able to predict its performance during design time.
- There are several ways to go about making such predictions.
- Analytic formulae use basic mathematical and statistical equations to estimate the performance of a system
  - This approach is highly abstract and uses simplified models. It is a quick and dirty approach but needs the expertise to model a system with a high level of detail while minimising assumptions.
  - Only needs design and component benchmarks. Component benchmarks may be available from vendors or previously known use cases.
- Simulation of a system by building an abstract model. You can use off-the-shelf or bespoke tools.
  - The used model contains many abstractions or simplifications.
  - Only needs a design and benchmarks.
  - With a simulation, you can explore variations and interactions and answer what-if questions.
- Monitoring involves the real system
  - It needs a real deployed system and anticipated workload mix.
  - Needs a monitoring framework to measure different aspects of the system, e.g., CPU utilisation and response time.
- The arrows illustrate the level of detail or effort required and the validity of the results.
  - While monitoring gives the most valid results, it is quite costly as you need a real system and people to test and collect data.
  - Whereas analytical formulae is a quick and dirty approach



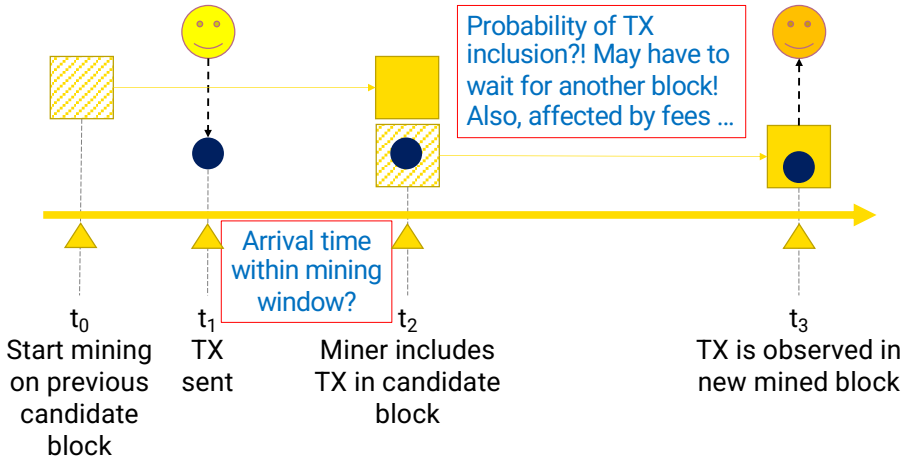
- The latency measures the time to complete a task on a system.
- In the blockchain context, it gives us an idea of transaction inclusion time in blockchains.
    - For some blockchains, this can be the same as the time to finality but for others, time to finality can be much higher than latency.
- This diagram illustrates the transaction inclusion time in public blockchains.
- Suppose a transaction arrives at time t_1 while the miner is already mining a block.
    - At t_0 miner starts mining the block for the current round, and it will take some time to mine, t_2 in figure (e.g., ~10 min Bitcoin).
- So, at best the transaction can only get included in the next block.
- If it gets included in the next block, the block will be mined at time t_3. The transaction sender or anyone else interested in the transaction can check its block inclusion at this time.
- As the time between 2 blocks is the inter-block time (~12 sec in Ethereum and ~10 min Bitcoin), the block-containing transaction will be confirmed only after inter-block time.
- We should also consider the delay as the transaction appeared between t_0 and t_2. At best, $t_1 = t_2$ so no delay. At most, $t_0 = t_1$ and the delay is one inter-block time. So, we consider the average (0 + 1)/2 inter-block time.
- Therefore, the expected time to include a newly arriving transaction in a block is 1.5 inter-block time
    - This is of course under the assumption transaction sender pays a very high transaction fee to get miners' attention to be included in the next block.

… Plus Extra Small Bits of Time



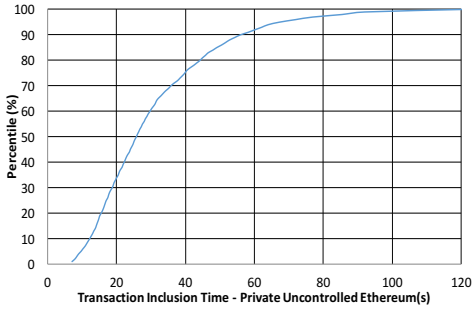
- In reality, there are various delays everywhere.
- While you may issue the transaction at t_1 it will take a bit of time to reach a large fraction of miners, $t_1' \gg t_1$.
- If the current block is mined by some other miner at time t_2, the announcement of that block will take some time to reach other miners, $t_2' > t_2$.
- Once that block is received only the miner can build the next block because it needs to include the parent block's hash in its block header. This also introduces a bit of a delay.
- Even if the block containing the transaction is built at time t_3, it will take a bit of time to the transaction sender or anyone else to validate its inclusion in the block, $t_3' > t_3$.
- Therefore, $t_3' - t_1$ is higher than $t_3 - t_1$ in the previous slide and likely be quite close to 1.9 to 2 inter-block times.
    - Note the additional delays, *TX propagation time, Block preparation time* and *Block propagation time*

… Also, Major Sources of Variation in Latency



- Also, there are other major sources of variation in latency.
- 10 min inter-block time in Bitcoin is average, and the actual time gap between 2 blocks can vary widely.
    - Sometimes 2 blocks may be built quite close to each other (< 2 min is not uncommon in Bitcoin) or very appear much apart (> 20 min is not uncommon either).
- So how much a transaction waits before being included in a block depends on the previous block's inter-block time.
- Once the transaction is included in that block needs to be mined, and the time mine the current block can also vary widely.
- Another source of variability is the transaction fee.
    - If the fee is high compared to other pending transactions, miners are likely to include the transaction in the next block. Otherwise, it may be substantially delayed.
    - There could also be a small fraction of altruistic miners that do not care about transaction fees and may include transactions based on their preferred order, e.g., first in best dress (aka FIFO).

High Variation of Latency



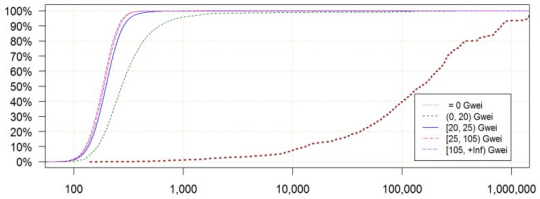


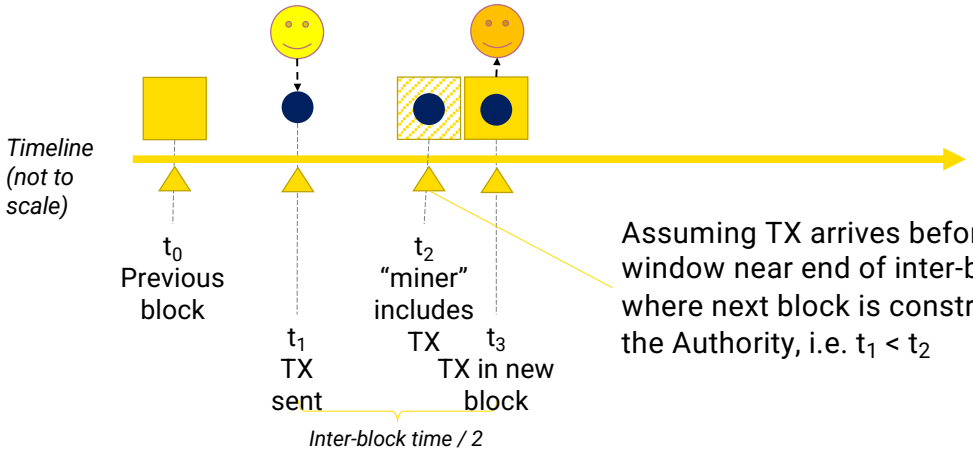
Figure 6: Commit delay (sec) for transactions based on gas price. Note logarithmic x axis.

N.B.: with 10 "confirmation blocks" adding a large "constant"-ish factor on the main cause of difference in latency variation

But with a lot of variation! Some depends on TX fee
Not 1.5 x inter-block time; in practice more like ~1.9 x inter-block time?
Design your system to cope with this to meet requirements

- The latency of blockchain can be a major drawback. As seen by the CDF in the left graph, there is considerable variability to include a transaction in a block. These results are from a private blockchain test.
- The 2nd graph is also a CDF from actual Ethereum transaction data. We can see the different gas prices lead to different distributions. Anyway, in all distributions there's a considerable variability.
- Given additional delays involved in propagating transactions and blocks, latency experienced by a transaction is more like 1.9 to 2 inter-block times.
- Although longer than in conventional systems, these transaction latencies may be acceptable for some use cases. You need to design your system to cope with this to meet requirements.
- It will still be important to be able to accurately predict system-level latency during the design phase, to assess the impact of this limitation on system requirements.

TX Inclusion with PoA in Private Blockchains



- The figure illustrates the latency for private blockchains with PoA (Proof of Authority) consensus like Hyperledger Fabric.
- Assuming the transaction arrives before the small window near the end of inter-block time where the next block is constructed by the Authority, i.e. $t_1 < t_2$
  - Given the same argument where the transaction may arrive just before the block is built or just after the previous block was built, the average latency is (0 + inter-block time) / 2.
- This can be tricky as inter-block time tends to be much smaller in private blockchains, but in this case, will have lower transaction latency anyway

How Architects Can Influence TX Latency?

Offer a TX fee that meets/exceeds miners' expectations

- Ethereum TX fee = Gas price x Gas used
- Gas price is user-defined

Choose smallest number of confirmation blocks that is "safe" for your application

Use another blockchain (private? or a public blockchain with lower latency)

- A blockchain with other consensus algorithms (PBFT, PoA, …)
  - Avoid Nakamoto consensus, so you can avoid confirmation blocks
- Reduce inter-block time

Work off-chain, e.g. use layer 2 solutions

- These are a few things an architect can do to deal with high transaction latency and its variability:
- The transaction fee is the only thing a user can control. Hence, offer a transaction fee that meets/exceeds miners' expectations. For this, you need an idea about transaction fees paid by similar transactions in the recent past.
  - Ethereum TX fee = Gas price x Gas used. As the gas price is user-defined that's what you can control.
  - Such estimates can be found on MetaMask and https://www.ethgasstation.info/
- Choose the smallest number of confirmation blocks that is "safe" for your application
  - While 6 confirmation blocks is the standard in Bitcoin, an application that wants to minimise risk further may choose to wait for 10-12 confirmation blocks.
- If you are hosting a DApp, then using another blockchain (private? or a public blockchain with lower latency) is an option.
  - E.g., as Ethereum transactions fees have substantially increased over the last 2-3 years, many NFTs have moved out of Ethereum into other platforms like https://flow.com/
  - If private, you can choose to use a blockchain with other consensus algorithms (PBFT, PoA, …). Essentially, try to avoid Nakamoto consensus, so you can avoid confirmation blocks.
  - Reducing the inter-block time is also an option. However, you cannot make it very small given the latency to propagate blocks across the globe.
  - Work off-chain, e.g. use layer 2 solutions such as a "state channel" design pattern.

Example – process Execution on Public Blockchain

A business process instance can be represented as a smart contract on a blockchain

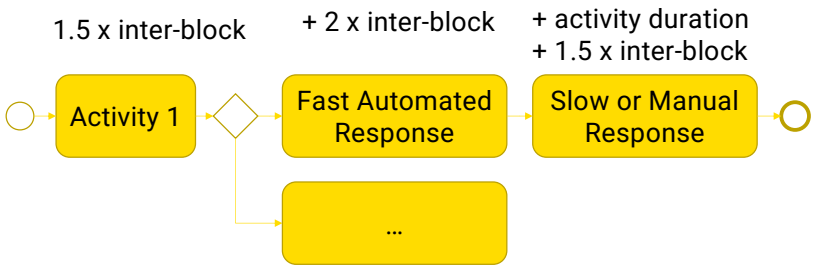
Each activity in process can be a different function in smart contract

An off-chain "trigger component" can check process state & invoke next activity in process

- If activity is automated: fast
- Otherwise: slow(er), e.g., in response to some manual activity

- Here is an example of a blockchain being used to facilitate business processes across multiple parties.
- A business process instance can be represented as a smart contract on a blockchain.
  - E.g., the figure is a BPMN diagram with 4 activities. Activity 1 is followed by 2 parallel activities (the diamond is called a gateway and may indicate parallel and exclusive execution options). Then the fast activity is followed by another activity before the business process terminates.
- Each activity in the business process can be a different function in the smart contract. You need to call these functions in the desired order to achieve proper business process exaction.
- An off-chain "trigger component" can check the process state and invoke the next activity in the process.
  - As smart contracts cannot interact directly with the external world, a trigger component should be used to connect the processes executing on blockchain to enterprise systems by acting as an agent for an organisation. The trigger manages keys, keeps track of the data payload in API calls, and can interact with external services including other databases or web services.
  - Generally, each organization has to deploy its trigger and at least one blockchain full node connected to the blockchain.
  - If activity is automated, it can either be executed on-chain or an external information system can immediately issue a transaction. In such cases, the delay is relatively low.
  - If the activity is handled manually there can be considerable delay in issuing a transaction to move the process to the next activity.

Latency of process Execution on Public Blockchain



- Quick & dirty estimate assuming no confirmation blocks
- Activity 1 arrives in the middle of some inter-block time window → 1.5 x inter-block time
- If activity response is very fast (in same inter-block time window), it'll still be too late to include in next block → 2 x inter-block time
- If activity response is very slow (comparable or larger than inter-block time), it'll arrive in the middle of some future inter-block time window

- Here's how we can get a quick and dirty estimate of the time to execute this business process using transactions.
- As Activity 1 is the 1st activity at least we can get the transaction that invokes or compete it within 1.5 x inter-block time (as per previous assumptions).
- Activity 1 is then followed up by the fast automated activity. However, this can only be performed once we know that the transaction for Activity 1 gets included in a block. This small delay is enough to get 2 times the inter-block delay as we discussed a few slides ago (as it is still too late to include in the next block).
- Then the final activity involves a manual task hence like to take at least 10 seconds. Such delay may be relatively higher compared to the inter-block time (10 s delay vs 12-sec inter-block time in Ethereum) and should be counted towards the final delay. Then once that transaction is issued, at best, we need to wait for another 1.5 x inter-block time to get that transaction included in a block.

Throughput

How many TXs can you process in a unit time?

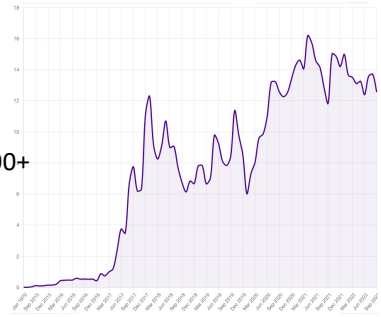
- e.g., as of 2017, Visa had a peak capacity to process 65,000 TPS

Public blockchains

- Public-permissionless, e.g., Bitcoin 3-7 & Ethereum 7-25 TPS
- Public-permissioned, e.g., Ripple & Stellar 1,000+, Hedera 10,000+ TPS

Permissioned blockchains (depends on setup)

- Private Ethereum – Quorum 50 TPS, Hyperledger Besu 500 TPS
- Hyperledger Fabric – 1,000+ (public TXs), 500+ (private TXs)
- R3 Corda – 170 TPS



Source: https://ethtps.info

You, as a user, are unlikely to create system overload!

- But, you need to understand trends in network utilisation & bottlenecks
  - e.g., as Dec 2017 ~ 1/3 TXs on Ethereum were for Cryptokitties

- Throughput is the no of transactions you can process in a unit of time.
  - E.g., as of 2017, Visa had a peak capacity to process 65,000 TPS. Typically, they process a few thousand TXs per second. With COVID, these numbers only went up…
  - TPS – TXs per Second
  - See https://www.visa.co.uk/dam/VCOM/download/corporate/media/visanet-technology/aboutvisafactsheet.pdf
- When Public-permissionless blockchains typically have poor TX throughput, e.g., Bitcoin has poor throughput of 3-7 TPS, and Ethereum processes 7-25 TPS. Ethereum 2l0 numbers are expected to go up
  - See https://medium.com/coinmonks/understanding-cryptocurrency-transaction-speeds-f9731fd93cb3
- The numbers are much better for public-permissioned blockchains. However, most of these blockchains either handle or claim numbers only for cryptocurrency transactions.
  - E.g., Ripple and Stellar clams 1,000+ TPS. Hedera claims more than 10,000 TPS.
- For permissioned blockchains, performance numbers depend on the test setup. The following numbers can be found in recent publications or vendor websites.
  - Private Ethereum network based on Quorum handles 50 TPS, while Hyperledger Besu has been shown to achieve 500 TPS
  - Hyperledger Fabric can process 1,000+ public TXs and 500+ private TXs involving private data collections (PDCs)
  - R3 Corda website claims 170 TPS
- A few 10s of TPS with the ability to handle a few 100s of TPS during peaks would be sufficient for most commercial applications. Also, be aware that the numbers heavily depend on the network set-up and types of transactions.
- You, as an individual, are unlikely to create system overload!
  - But, you need to understand trends in network utilisation and bottlenecks, as they will impact your transaction.
  - E.g. As of Dec 2017 ~ 1/3 TXs on Ethereum were for Cryptokitties. Forsage is a pyramid scheme that also congested the network at times.

Block Configuration

Block size & difficulty determine TX throughput

No of TXs per second = No of TXs in block / Inter-block time

Increase block size

- Increase block size in terms of MB or gas limit
- Larger blocks contain more TXs → Increase throughput, more TX fee for miners
- Needs more time to replicate across the network
  - More competing blocks → More forks → Needs more confirmation blocks
- Block takes time to fill up
  - Empty/small blocks – Also, happen when mining reward >> TX fees
  - Wait till block get filled → Increase inter-block time → Reduce throughput

- Block configuration affects system performance.
- Given a sufficient no of transactions are waiting to be included in a block, transaction throughput (i.e., no of transactions included in a block within unit time) depends on block size and average inter-block time (i.e., time between 2 blocks).
- Therefore, it looks natural to increase the block size while reducing inter-block time. But this is not straightforward.
- Block size can be increased by increasing block size in MB (e.g., Bitcoin) or gas limit (e.g., Ethereum).
- When the block size is high, we can include more TXs per block. Therefore, TX throughput is increased. Also, miner can collect more TX fees per block.
- However, large blocks need more time to replicate across the network, e.g., think of downloading a large file.
  - While one block slowly propagates another miner who didn't hear about that block, may build a competing block and propagates that.
  - This Increases frequency of forks, i.e., more competing blocks.
  - As as per Nakamoto consensus we need to wait till longest chain of blocks emerge, which will take more time to appear due to many competing blocks (or may not be ever). Therefore, we need more confirmation blocks before accepting a TX as final with high probability.
- Large blocks also take time to fill up if not enough transactions are waiting in the transaction pool.
  - This will results in empty or small block.
  - Empty/small blocks may also happen when mining reward is very high compared to TX fees. So rather than bothering to include TXs miners may start block building as soon as previous block is received
  - Empty blocks, sometimes called "1-transaction blocks," include only the coinbase reward and forgo additional transaction fees
  - Alternatively, if we wait for the block to be fill it increases the inter-block time, reducing throughput

Block Configuration – Cont.

Reduce inter-block time

- Increases throughput
  - Can't be lower than block propagation time
- Reducing block difficulty
  - Reduce mining effort → More forks → Needs more confirmation blocks
  - A few large miners computing power may be enough to control the network
    - Risk of 51% attack increases

- The other option is to reduce inter-block time. When it's reduced we can increase throughput as per the previous equation.
- However, it can't be too small such that it's close to or shorter than the block propagation time across the network. Then new blocks get generated before the previous block is known by most of the network.
- To reduce inter-block time, we need to reduce block difficulty, e.g., by reducing the no of leading zero's to be found in the PoW result.
- This is good from the miners' point of view as it reduces the mining effort, i.e., computational complexity.
- However, that also means more miners will be able to solve the puzzle resulting in multiple competing blocks. Therefore, this increases the frequency of forks.
- As discussed earlier, more competing blocks mean we need to wait for more confirmation blocks before accepting a transaction as final.
- A bigger concern is a few large miners may be able to assemble enough computing power to gain control of the network. This increases the risk of the 51% attack.
  - 51% attack – This is the ability of one or a set of colluding parties to control more than 50% of the computing power in the network to control what blocks to build.

How Architects Can Influence TX Throughput

Must use a specific public blockchain

- Focus on how you use blockchain & other areas of application architecture
  - Reduce on-chain data
  - Do more work off-chain, e.g. use “state channel” design pattern

Use another blockchain (private? or a public blockchain with high throughput?)

- Blockchain with other consensus algorithms (PBFT, PoA, …)
- Sharding, DAGs
- Configure blockchain to use a larger block size or smaller inter-block time
- If you don’t need exchange of digital assets, consider networks of blockchains

- Here are a couple of options available for the architect to increase the transaction throughput.
- If you must use a specific public blockchain, then focus on how you use blockchain and other areas of application architecture, e.g.,
  - Reduce on-chain data that way you may be able to reduce the number of transaction of be able to pay more for a smaller transaction.
  - Do more work off-chain, e.g. use “state channel” design pattern. This was we reduce transactions and smart contract complexity which reduced gas used
- If you are going to deploy your blockchain, use another blockchain (private? or a public blockchain with high throughput?)
  - E.g., we can use a blockchain with other consensus algorithms (PBFT, PoA, …) or ledger data structures like sharding and DAGs.
  - Configure blockchain to use a larger block size or smaller inter-block time. Remember this need to be done carefully as discussed in the previous slide.
  - If you don’t need an exchange of digital assets, e.g., traceability or ownership recording only, then consider networks of blockchains. Remember pair-wise ledgers from 1st session?

Exercise

From the following 2 blockchain configurations, which one has the highest throughput?

Configuration	Maximum block size	Average inter-block time
A	1,000	15 sec
B	500	10 sec

- Throughput of A = 1000/15
- Throughput of B = 500/10
- So A > B

Agenda

Functional suitability

Performance efficiency

- Time behaviour; latency and throughput

Reliability; availability and recoverability

Reliability Characteristics

Reliability	Degree to which a system, product, or component performs specified functions under specified conditions for a specified period
Availability	Degree to which a system is operational & accessible when required for use
Recoverability	Degree to which, in the event of an interruption or a failure, a system can recover data directly affected & re-establish desired state of system
Maturity	Degree to which a system meets needs for reliability under normal operation
Fault-Tolerance	Degree to which a system operates as intended despite the presence of hardware or software faults

- These are the reliability characteristics of the ISO quality attributes.

Availability

Is system operational & accessible when required for use?

Usually measured in terms of number of 9s

- 99.9% (8:45h downtime per year), 99.99% (52.5 min), 99.999% (5.25 min)

Ledger is highly available for reads

- Can read from any of the 10s to 1,000s of nodes in the network
- Some ledgers may prune old state, yet active state must still be in ledger

Ledger has low availability for writes

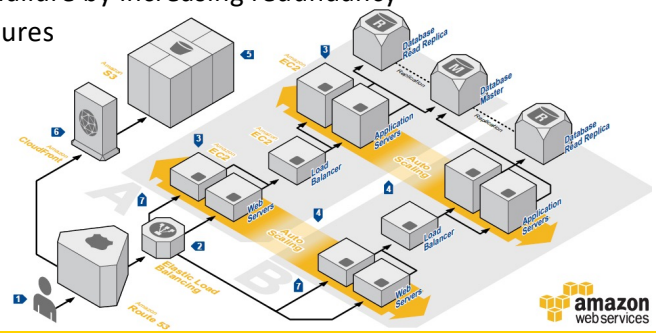
- Sequential block building with limited block size
- Latency varies from seconds to hours
- TX could even get dropped

- ISO 25010 defines availability as the degree to which a system, product or component is operational and accessible when required for use.
- Availability is usually measured in terms of the number of 9s.
    - E.g., availability of 99.9% means a system can't be down for more than 8:45h per year.
    - Similarly, 99.99% means less than 52 min and 30-sec downtime annually. 99.999% means less than 5 min and 15-sec downtime annually.
- As a blockchain network has many nodes (10s to 1,000s), and the ledger can be read from any of them (in permissioned networks, it will depend on no of nodes maintaining the ledger), the leader is highly available for reads.
    - Some ledgers may prune the old state (e.g., even Bitcoin suggests pruning used UTXOs), but the active state must still be in the ledger. In that case, you can still read the active state but not the old/used state, which may be maintained as an archive.
- We consider the ledger has low availability for writes due to the difficulty of getting a TX included in a block, i.e., it's not necessarily accessible to write when you need (the network will accept your TX as far as it's valid. However, it doesn't include TX in a block immediately).
    - Sequential block building with one winning miner/validator at a time and limited block size limit TX inclusion in a block.
    - Due to varying TX fees, inter-block time, and zero TX blocks, latency experienced by a TX could vary from seconds to hours, or even be dropped.

How Architects Can Influence Availability?

Use typical design strategies to enhance availability of components that interacts with blockchain

- Use highly available components & connectors
- Deploy them on reliable hardware & networks
- Eliminate single points of failure by increasing redundancy
- Detect & recover from failures



- Blockchain has the highest availability. Hence, the focus should be on increasing the availability of components that interacts with the blockchain.
- Architects can use typical availability-increasing design strategies to enhance the availability of components that interacts with blockchain, e.g.,
  - Use highly available components and connectors.
  - Deploy them on reliable hardware and networks. Overall availability is the multiplication of the unavailability of both components/connectors and underlying hardware.
  - Eliminate single points of failure by increasing redundancy. Such a design would require load balancing, failover monitoring and routing. Also, stateless server components can enable a new component to resume work without establishing the state before failure.
  - We also need mechanisms to detect (e.g., pinging a node) and recover from failures, e.g., hot backup or failover servers.
- This diagram illustrates a fault-tolerant design for cloud-based services.

Recoverability

Can system recover affected data & re-establish system's state in the event of an interruption or a failure?

Blockchain is likely to recover

- Very high redundancy
- Nodes can sync from where they left off
- Consensus mechanisms are designed to autonomically recover nodes & ledger consistency

In worst case in a public blockchain, there might be a hard fork

- In a hard fork, different subsets of miners run conflicting consensus protocol rules, & blocks, TXs, & ledger state don't agree across subsets

Focus on other parts of application architecture

- Recoverability is the degree to which, in the event of an interruption or a failure, a product or system can recover data directly affected and re-establish the desired state of the system.
- Blockchain is likely to recover as it is designed to tolerate failures and malicious nodes, e.g.,
  - There is very high redundancy.
  - As far are some nodes are running other nodes can sync from where they left off.
  - In case of a conflict in that process, consensus mechanisms are designed to autonomically resolve conflicts and recover nodes and ledger consistency.
- In the worst case in a public blockchain, there might be a hard fork in the ledger.
  - A hard fork occurs when different subsets of miners/validators run different versions of consensus protocol rules where blocks and transactions valid in one set of miners are not valid in another set. This splits a blockchain network where different network segments different multiple ledger states.
  - After the DAO attacks miners who believe in code is law did not upgrade the protocol to reclaim lost Ether. Those miners split their network as Ethereum Classic and those who applied to upgrade now run Ethereum.
- Typically, recoverability will mostly be a concern for other parts of application architecture.

Aborting a Transaction in Ethereum

If your TX is in TX pool (aka mempool) can you abort it?

- Changed your mind, something is wrong, or taking too long to commit

To speed up TX inclusion

- Reissue same TX, from same address with a higher TX fee
- E.g., for bitcoin same input & output UTXOs with a higher TX fee

To cancel out previous TX

- Issue a competing null TX with same address with a higher TX fee
- E.g., send 0 Ether to yourself or invoke a SC to raise an exception while setting same address, nonce, & high gas price

Not guaranteed to work

- Previous TX may just get included
- Altruistic mining & miners going behind block reward over TX fees

- Conventional transaction processing systems allow a transaction to be aborted while it is pending or being processed.
- The related question in blockchains is if you send a TX into the TX pool (aka mempool), and it's not yet included in a block, can you abort it?
    - You may want to do this due to your changing mind, something is wrong with the pending transaction, or it is taking too long to commit so either you need to speed it up or give up.
- To speed up transaction inclusion we can reissue the same transaction, from the same address with a higher transaction fee.
    - E.g., we can issue the same transaction with the same input and output UTXOs with a higher transaction fee. A higher transaction fee means one of the output UTXOs (e.g., the balance returned to you) needs to be reduced.
    - The idea is that miners will first include transactions with high fees making future use of the same input UTXOs invalid aborting the previous transaction.
- To cancel out the previous transaction, we can send a completed transaction with the same address but with a significantly higher transaction fee.
    - E.g., we can send 0 Ether to you or invoke a smart contract to raise an exception while setting the same address, nonce, and high gas price. The idea is that miners will drop the previous transaction to gain the advantage of the high gas fee offered.
        - Ethereum transactions have a nonce that is increased with every transaction (this nonce is different from the nonce in the block header). Miners will consider a reused nonce as being "outdated" and discard those transactions.
    - In Bitcoin, we can use the same input UTXO and return them to your address while setting a high transaction fee.
- There's no guarantee that 2nd transaction will replace the previous one:
    - It may just get included as your 2nd transaction started to propagate across the blockchain.
    - Few altruistic miners don't care so much about the transaction fee or they may be motivated by block creation rewards and randomly pick transactions to include in a block
- A higher fee means it has a different hash value, so will be seen as a "different" TX

Maturity

Can system maintain reliability under normal operation

- Reliability is about continuity of correct service
- Essentially on ability to maintain correct service over a long period

Ability to consistently provide service depends on both blockchain & components attached to it

- Previous discussion about availability for blockchain-based applications applies here too

- Maturity is the degree to which a system, product, or component meets the needs for reliability under normal operation.
    - Maturity is a confusing name and ISO standard has serval such names.
- It is essentially about whether a system can maintain reliability under normal operation.
    - As reliability is about the continuity of correct service, this quality is essentially on the ability to maintain correct service over a long period of time
- The ability to consistently provide service depends on both blockchain and components attached to it. Hence, we should consider previous discussion about the availability for blockchain-based applications applies.

Fault-Tolerance

Can system operate as intended despite hardware or software faults?

Blockchains tolerate faults

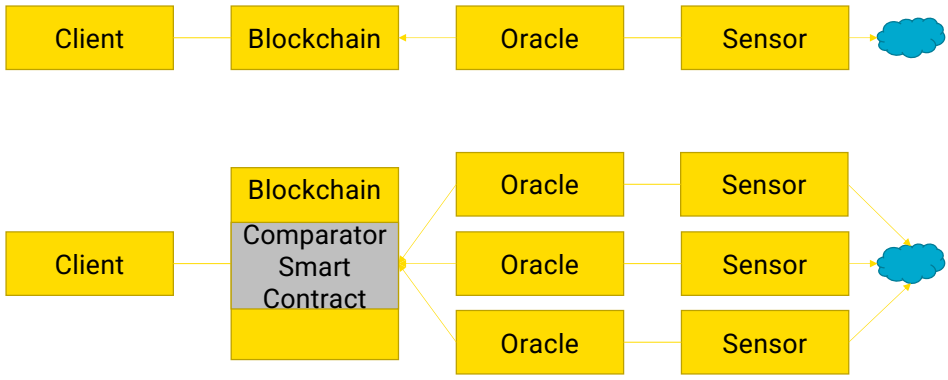
- Hardware faults of miners → Others will keep working
- Software faults too, if there's enough diversity of mining software
  - Different versions & implementations, e.g., Geth & Parity nodes in Ethereum
  - In worst case, might get a hard fork

For blockchain applications

- Use typical design strategies for fault tolerance (redundancy, monitor, detect, isolate recover) for other parts of architecture
- Smart contracts aren't normally fault-tolerant for software faults
- Smart contracts can support fault tolerance logic for off-chain components…

- Fault tolerance is the degree to which a system, product, or component operates as intended despite the presence of hardware or software faults.
- It is essentially about whether a system can continue to operate as intended despite failures in hardware or software.
- A bit of terminology:
  - Error – System state that could lead to a service failure
  - Fault – Adjudged or hypothesized cause of an error
  - Fault tolerance – Avoid service failures in the presence of faults
- Blockchains are pretty good at tolerating faults, e.g.,
  - Due to the high number of miners/validators hardware faults of miners is not a problem as others will keep working. While system-wide hardware failures are unlikely network failure is possible.
  - Software faults too can be tolerated if there's enough diversity in mining software.
    - Ideally, not all miners should run the same version of the software as a bug in one means a bug in all miners.
    - Fault-tolerant computing advocates miners running different versions and implementations, e.g., Geth and Parity nodes in Ethereum (Geth is based on GO while Party is based on Rust language).
    - However, ~80% of Ethereum nodes are Geth, meaning the software is not very diverse.
    - In the worst case, software failures might lead to a hard fork. Both Bitcoin and Ethereum have had similar incidents where upgrades to client/miner software had made certain block invalid creating an alternative branch in the chain of blocks.
- For blockchain applications, we can use typical design strategies for fault tolerance (redundancy, monitor, detect, isolate recover) for other parts of the architecture. E.g., the diagram in the previous slide was a fault-tolerant design.
  - Smart contracts aren't normally fault-tolerant for software faults as the same piece of code runs across the entire network. So a smart contract has an issue, it's everywhere. Due to immutability, it's not straightforward to fix and redeploy.
  - Smart contracts can support fault tolerance logic for off-chain components, see next slide

Oracle Faults



Use on-chain M-of-N voting for redundant oracles

- An oracle is a single point of failure for a blockchain-based application. Hence, if the oracle is down, we can continue the application.
- A decentralised oracle can address this problem as we have multiple oracle instances.
    - We can tolerate hardware failures.
    - To truly tolerate software failures on-chain/off-chain oracle code must be diverse.
    - Also, to delay with sensor failures, we need have redundant sensors too.

Question

Mark True or False for each the following statements

	True	False
Formal specification & verification ensure a smart contract is bug-free		✓
Availability of a blockchain-based application solely relies on the availability of the blockchain platform		✓
We should focus on enhancing the reliability of components that interacts with the blockchain	✓	
By offering a transaction free higher than any of the pending transactions, we can get our transaction included in a block within 1.5 x inter-block time		✓

1. Formal specification & verification ensure a smart contract is bug-free. – They go a long way in ensuring there are no bugs. But they can't give you a guarantee.
2. Availability of a blockchain-based application solely relies on the availability of the blockchain platform. – Other components also impact the availability such as an Oracle
3. We should focus on enhancing the reliability of components that interacts with the blockchain. – Blockchain is quite reliable (except for the diversity of blockchain client software). So, the focus should be mainly on other components that interact with it
4. By offering a transaction free higher than any of the pending transactions, we can get our transaction included in a block within 1.5 x inter-block time. – 1.5 x inter-block time is the best case. There's no guarantee that it will be the case even if you pay a very high transaction fee given altruistic mining and miners that go behind block reward over transaction fees.